

Theory of Programming Languages

8. Recursive Types

Kihong Heo



Motivation

- Typed λ -calculus with general recursion
 - System PCF
 - Term-level recursion
- General typing mechanism for handling inductive types and general recursion
 - System FPC
 - Type-level recursion

System PCF

- Introduced by Plotkin (1977)
 - “LCF considered as a programming language”, Theoretical Computer Science, 1977
- Typed λ -calculus with general recursion (i.e., term-level recursion)
 - A typed program may not terminate
- PCF: Programming Computable Functions

Syntax

- Simply typed λ -calculus + natural numbers + fixpoint operators

$$\begin{array}{lll} \text{Term} & E & \rightarrow x \mid \lambda x:\tau.E \mid E E \\ & & \mid 0 \mid \mathbf{s}(E) \mid \mathbf{ifz} E E x.E \\ & & \mid \mathbf{fix}_\tau x.E \\ \text{Type} & \tau & \rightarrow \mathbf{nat} \mid \tau \rightarrow \tau \\ \text{Value} & v & \rightarrow 0 \mid \mathbf{s}(v) \mid \lambda x:\tau.E \end{array}$$

Static Semantics

- Variable & function

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}$$

$$\frac{\Gamma + x : \tau_1 \vdash E : \tau_2}{\Gamma \vdash \lambda x : \tau_1. E : \tau_1 \rightarrow \tau_2}$$

$$\frac{\Gamma \vdash E_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash E_2 : \tau_1}{\Gamma \vdash E_1 E_2 : \tau_2}$$

- Natural number

$$\frac{}{\Gamma \vdash 0 : \text{nat}}$$

$$\frac{\Gamma \vdash E : \text{nat}}{\Gamma \vdash \text{s}(E) : \text{nat}}$$

$$\frac{\Gamma \vdash E_0 : \text{nat} \quad \Gamma \vdash E_1 : \tau \quad \Gamma + x : \mathbf{x} \vdash E_2 : \tau}{\Gamma \vdash \text{ifz } E_1 E_2 x.E_3 : \tau}$$

- General recursion

$$\frac{\Gamma + x : \tau \vdash E : \tau}{\Gamma \vdash \text{fix}_\tau x.E : \tau}$$

Dynamic Semantics

- Evaluation contexts

$$\text{Ctx } K \rightarrow \begin{array}{l} [] \mid K \ E \mid v \ E \\ \mid \text{ s}(K) \mid \text{ifz } K \ E \ x.E \end{array}$$

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

- β -reductions

$$\frac{}{\text{ifz } 0 \ E_1 \ x.E_2 \rightarrow E_1} \quad \frac{}{\text{ifz } \text{s}(E) \ E_1 \ x.E_2 \rightarrow E_2\{E/x\}}$$

$$\frac{}{\text{fix}_\tau \ x.E \rightarrow E\{(\text{fix}_\tau \ x.E)/x\}}$$

Exercise: Fixed Point (Revisited)

$$\text{fact}(N) = \text{if } N = 0 ? 1 : N * \text{fact}(N - 1)$$
$$\text{fact} = \lambda N. \text{if } N = 0 ? 1 : N * \text{fact}(N - 1)$$

fact is the least fixed point of F where $F(X) = \lambda N. \text{if } N = 0 ? 1 : N * X(N - 1)$

$$\therefore \text{fact} = \text{fix}(\lambda X. (\lambda N. \text{if } N = 0 ? 1 : N * X(N - 1)))$$

Exercise

- Double
- Plus
- Factorial
- Infinite loop

```
let rec double x =  
  match x with  
  | Zero -> Zero  
  | Succ y -> Succ (Succ (double y))
```

```
let rec plus x y =  
  match x with  
  | Zero -> y  
  | Succ x' -> Succ (plus x' y)
```

```
let rec factorial x =  
  match x with  
  | Zero -> Succ Zero  
  | Succ y -> times x (factorial y)
```

```
let rec f x = f x
```

System FPC

- Introduced by Plotkin (1985)
 - “Lectures on predomains and partial functions”
- Provide a general mechanism to define inductive types
- FPC: Fix Point Calculus

Exercise

- Define the following types using unit, function, product, and sum types

- How to define the Boolean type?

```
type boolean = True | False
```

$\text{boolean} = \text{unit} + \text{unit}$

- How to define the Nat type?

```
type nat = Zero | Succ of nat
```

$\text{nat} = \text{unit} + \text{nat}$

- How to define the Nat List type?

```
type natlist = Nil | Cons of nat * natlist
```

$\text{nat list} = \text{unit} + \text{nat} \times \text{nat list}$

Recursive Type

- How to define a type with infinitely many cases?
 - E.g., $\text{nat} = \text{unit} + \text{unit} + \dots = \text{unit} + \text{nat}$
- Idea: type equation $\alpha = \tau$ and solution $\mu\alpha. \tau$
 - I.e., $\mu\alpha. \tau = \tau\{\mu\alpha. \tau / \alpha\}$
- Examples:
 - $\text{nat} = \mu\alpha. \text{unit} + \alpha$
 - $\text{nat list} = \mu\alpha. \text{unit} + \text{nat} \times \alpha$

Static Semantics

- Language: λ -calculus with recursive types

$$\begin{array}{lcl} \tau & \rightarrow & \dots \\ | & & \alpha \quad \text{Type variable} \\ | & & \mu\alpha.\tau \quad \text{Solution of } \alpha = \tau \end{array}$$

- Type rules

$$\frac{\Gamma \vdash E : \tau\{\mu\alpha.\tau/\alpha\}}{\Gamma \vdash E : \mu\alpha.\tau} \mu I \qquad \frac{\Gamma \vdash E : \mu\alpha.\tau}{\Gamma \vdash E : \tau\{\mu\alpha.\tau/\alpha\}} \mu E$$

Exercise: Nat List (1)

- Type: $\text{nat list} = \mu\alpha.\text{unit} + (\text{nat} \times \alpha)$
- nil:

$$\frac{\frac{\Gamma \vdash () : \text{unit}}{\Gamma \vdash \text{inl } () : \text{unit} + (\text{nat} \times (\mu\alpha.\text{unit} + (\text{nat} \times \alpha)))}}{\Gamma \vdash \text{inl } () : \mu\alpha.\text{unit} + (\text{nat} \times \alpha)}$$

- cons:

$$\frac{\dots}{\Gamma \vdash \lambda n:\text{nat}.\lambda xs:\text{nat list}.\text{inr}(n, xs) : \mu\alpha.\text{unit} + (\text{nat} \times \alpha)}$$

Exercise: Nat List (2)

- Type: $\text{nat list} = \mu\alpha.\text{unit} + (\text{nat} \times \alpha)$
- `hd`:

$$\frac{\dots}{\Gamma \vdash \lambda l:\text{nat list}.\text{case } l \text{ of } \text{inl } u \Rightarrow 0 \mid \text{inr } p \Rightarrow p.1 : \text{nat}}$$

- `tl`:

$$\frac{\dots}{\Gamma \vdash \lambda l:\text{nat list}.\text{case } l \text{ of } \text{inl } u \Rightarrow l \mid \text{inr } p \Rightarrow p.2 : \text{nat list}}$$

Exercise: Nat List (3)

- Type: $\text{nat list} = \mu\alpha.\text{unit} + (\text{nat} \times \alpha)$
- isnil:

$$\frac{\dots}{\Gamma \vdash \lambda l:\text{nat list}.\text{case } l \text{ of } \text{inl } u \Rightarrow \text{true} \mid \text{inr } p \Rightarrow \text{false} : \text{bool}}$$

- sumlist:

$$\frac{\dots}{\Gamma \vdash \text{fix}(\lambda f:\text{nat list} \rightarrow \text{nat}.\lambda l:\text{nat list}.\text{case } l \text{ of } \text{inl } u \Rightarrow 0 \mid \text{inr } p \Rightarrow p.1 + f(p.2)) : \text{nat list} \rightarrow \text{nat}}$$

Exercise: Fix Point Combinator

- Y combinator: $\text{fix} = \lambda f. (\lambda x. f(x\ x)) (\lambda x. f(x\ x))$
- Typed version: $\text{fix}_\tau = \lambda f : \tau \rightarrow \tau. (\lambda x : (\mu \alpha. \alpha \rightarrow \tau). f\ (x\ x))\ (\lambda x : (\mu \alpha. \alpha \rightarrow \tau). f\ (x\ x))$
- Divergence: $\Omega = (\lambda x. x\ x) (\lambda x. x\ x) : \tau$
- Strong normalization does not hold anymore: well-typed terms may not terminate

Summary

- System PCF: typed λ -calculus + term-level recursion
 - Fixpoint combinator as a primitive
 - General recursion supported
 - Strong normalization is lost
- System FPC: typed λ -calculus + type-level recursion
 - Recursive types as type-level fixed points
 - Term-level fixed points (general recursion) definable